

VALO — What It Is & Why It Matters

Plain-Language Introduction

Version: V5.0 — Validation

Phase

Date: May 15, 2026

Classification: Confidential — NDA Required

The Problem

Critical infrastructure AI cannot be proven safe using traditional testing. VALO is a three-layer system designed to provide mathematical proof of safety, not statistical confidence.

Testing vs. Proof

Traditional Testing	VALO Formal Specification
"We ran N scenarios and nothing broke."	"We specify ALL scenarios must be safe."
"Probably safe"	"Provably safe" — pending UiS verification

Three-Layer Architecture

Layer	Component	Language	Function
L1	Guardian	Rust	Safety decision gate — ACCEPT/REJECT in 43ns. 0 unsafe blocks. Only formally specified layer.
L2	Orchestrator	Python	Logging, routing, authentication, SHA-256 WORM audit log. Cannot override L1.
L3	Context Engine	Rust/Python	Source reliability tracking, distrust escalation (L0 Trusted → L4 Total Halt).

Architectural guarantee: L2 and L3 cannot override L1's decision. If L1 says REJECT, the system halts. This is enforced architecturally, not by convention.

Current Status

TLA+ specification	Complete — 5 states, 6 transitions, 4 invariants	✓
L1 Rust implementation	180 lines, 0 unsafe blocks	✓
Security audit	0 critical findings (May 2026)	✓
L3 distrust matrix	5 trust levels (L0 Trusted → L4 Halt)	✓

Model checker (TLC)	Pending — UiS executes independently	→
Empirical test suite	1M cases designed — execution pending	→

GitHub Repository Structure

Repository: Valo-Research-Geoup/valo-v5-core (private)

/		
VALIDATION.md	Formal validation specification and scope	
WHITEPAPER.md	Technical whitepaper (Markdown)	
SOVEREIGNTY.md	IP sovereignty and confidentiality declaration	
Dockerfile	Container build definition	
simulate.py	Infrastructure simulation runner	
simulate_vaig.py	VAIG (AI inference guard) simulation	
/l1-guardian/		
src/main.rs	L1 Guardian entry point — Rust	
src/lib.rs	Core safety logic — ACCEPT/REJECT gate	
src/valo_frame.rs	64-byte cache-aligned ValoFrame struct	
src/validation_logic.rs	Safety measurement logic	
src/crc32c.rs	CRC32C integrity verification	
src/server.rs	Socket server — UDS/TCP transport	
/l2-orchestrator/		
ai_gateway.py	AI model interface and routing	
src/auth.py	Authentication layer	
src/worm_log.py	SHA-256 hash-chained WORM audit log	
src/propagator.py	Frame propagation and transport	
/l3-context/		
src/context_engine.rs	Source reliability engine — Rust	
src/distrust.rs	Distrust escalation matrix (L0–L4)	
src/feeds.rs	External data feed management	
/formal-verification/		
ValoStateMachine.tla	Primary TLA+ specification — L1 state machine	
ValoStateMachine.cfg	TLC model checker configuration	
valo_v5.tla	Extended TLA+ specification	
/docs/		
Valo_Research_Group__WP_Valo_5_0_L OCKED.pdf	Locked whitepaper PDF	
/sidecar/		
vaig.py	VAIG — AI inference guard sidecar	
client.py	Test client	

"We code integrity, not probability." — VALO | njaal.solland@gmail.com | Confidential